AI-Assisted Parallelization of bzip2 Compression

Final Project — Parallel Programming, Spring 2026

Lin Ching-Yun, Hsieh Tien-Hua, Cheng Yu-Shiang
Department of Computer Science and Information Engineering
National Taiwan University
{b12902116, b11902088, b12902025}@ntu.edu.tw

***Abstract*—Data compression is a critical component of high-performance computing, with algorithms like bzip2 prized for their high compression ratios. bzip2 internally processes data in blocks, making it a potential candidate for parallelization. However, effectively parallelizing this process requires careful management of output ordering, synchronization overhead, and I/O bottlenecks. This paper explores the efficacy of Large Language Models (LLMs) in automating the parallelization of a sequential bzip2 compressor (pbzx). We evaluate an AI-assisted development workflow across three progressive stages of guidance: naive prompting, constraint-guided design, and profiling-guided optimization. By comparing these AI-generated implementations against a sequential baseline and two established parallel references (lbzip2 and pbzip2), we analyze how varying levels of system-level feedback impact the correctness, speedup, and parallel efficiency of AI-generated C code. Our results show that profiling-guided optimization achieves the best performance, reaching throughput parity with hand-optimized tools, while strict constraint-guided prompting inadvertently introduces structural bottlenecks.**

***Index Terms*—Data compression, bzip2, parallel programming, LLM code generation, AI-assisted development, performance profiling**

Project Category: ☒ **A** (Self-chosen topic) ☐ **B** (AI-agent code optimization)

I. Introduction

Large language models may help identify parallelizable components, generate multithreaded code, suggest pipeline designs, and optimize performance. However, AI may not fully understand the target machine, runtime behavior, synchronization costs, or the actual performance bottlenecks after parallelization. Therefore, we guided the AI agent using profiling results and system-level feedback, and evaluated how this feedback affected correctness and performance.

The objective of this project is to explore different ways of using AI to assist in parallelizing a bzip2-like compression system, and to evaluate whether AI-generated or AI-assisted parallel versions can improve performance while preserving correctness. Rather than simply dividing blocks among threads, effective parallelization requires overcoming distinct challenges, such as managing I/O bottlenecks, minimizing lock contention, and maintaining strict output ordering. We contribute an evaluation of a multi-stage AI-agent workflow, demonstrating that shifting from naive generation to bottleneck-aware, profiling-driven prompting allows LLMs to successfully navigate the complexities of parallel code optimization.

II. Background and Related Work

A. The bzip2 Algorithm

bzip2 processes data in independent blocks, so block-level parallelism is possible. Since different data blocks can be compressed independently, multiple blocks may be assigned to different worker threads and processed in parallel. In addition, block size may affect both compression ratio and parallel efficiency.

B. Existing Parallel bzip2 Implementations

Two mature open-source tools parallelize bzip2 compression and serve as external references in our evaluation.

1) lbzip2: lbzip2 is an algorithmically optimized reference: it emits byte-compatible `.bz2` output but, unlike pbzx, does not link libbz2. It replaces bzip2's comparison-based blocksort with its own libdivsufsort-style suffix-array BWT (`divbwt`), computing the identical transform with far less work, and parallelizes through a hand-built `pthread` pipeline rather than OpenMP fork/join. It was the fastest *per-core* implementation in our sweeps; profiling showed it to be branch-prediction-bound and already near the hardware ceiling, so we use it unmodified.

2) pbzip2: pbzip2 is the closest reference to our own design: like pbzx, it is a block-parallel front-end *on top of* stock libbz2, so it shares bzip2's codec and thus the same per-core BWT cost and compression ratio. It differs only in how work is distributed — a hand-built POSIX-threads producer/worker/writer pipeline with order-preserving output, in place of pbzx's OpenMP fork/join. It is, in effect, the hand-coded ancestor of the same block-parallel idea our AI-assisted versions express through OpenMP.

III. Methodology

Instead of only asking AI to "make the code faster," we designed a structured three-stage workflow to guide the agent step by step. The project isolates the development of each AI-assisted implementation on dedicated Git branches stemming from the main baseline.

A. Sequential Baseline and AI-Agent Setup

To isolate the effects of our parallelization strategies, we start from pbzx, a custom C11 baseline that sequentially compresses blocks using vendored libbz2. All AI-assisted work is carried out by the Claude Code agent, which operates autonomously within isolated Git worktrees — one per stage — branched from this baseline, keeping each implementation independent and reproducible.

B. Stage 1: Naive AI Parallelization

In the first stage, we provided the agent with the sequential bzip2 compression code and asked it to parallelize the program. The agent received a minimal prompt without hints regarding the parallelization strategy. This stage tested whether AI can independently identify the parallelizable parts of the program.

C. Stage 2: Constraint-Guided AI Parallelization

In the second stage, we provided the agent with explicit design requirements and environment-awareness. Crucially, we treated the underlying hardware as a physical constraint the agent must optimize for: we granted the agent permission to execute system discovery commands (e.g., `lscpu`, `nproc`, `free -h`) to read the machine's core topology, NUMA nodes, memory limits, and cache sizes. Alongside this hardware context, the agent was structurally prompted to use block-level parallelism, assign unique block IDs, use worker threads to compress independently, and employ a writer thread to maintain original output order. This stage tested whether combining hardware-awareness with explicit system design constraints helps AI generate more correct and highly optimized parallel code.

D. Stage 3: Profiling-Guided Optimization

In the third stage, the agent again parallelized the sequential baseline from scratch, but this time with access to profiling tools. We ran its parallel version and collected profiling information, such as CPU utilization, runtime breakdown, I/O waiting time, lock contention, memory usage, thread idle time, and scalability under different thread counts, then fed this empirical bottleneck data back into the agent's context so it could iterate. This stage tested whether profiling feedback can help AI move from superficial code changes to bottleneck-driven optimization.

IV. Experimental Setup

A. Hardware and Software Environment

The experiments were conducted on a dual-socket server running Ubuntu 24.04 Server, equipped with two Intel Xeon Platinum 8352V processors (Ice Lake architecture) operating at a 2.10 GHz base clock (up to 3.50 GHz turbo). The system provides a total of 72 physical cores (36 per socket) and 144 logical threads via Intel Hyper-Threading, partitioned into two NUMA nodes. The cache hierarchy consists of 48 KB L1d, 32 KB L1i, and 1.25 MB L2 cache per physical core, backed by a shared 54 MB L3 cache per socket, with support for AVX-512 instruction set extensions.

The target software environment utilizes C11 with pthreads or OpenMP, orchestrated by a Python-based benchmarking harness. Profiling data is gathered using hardware performance counters via `perf stat`, call-graph hotspots via `perf record`, and resource monitoring via GNU `/usr/bin/time -v`. We evaluate both system performance and AI effectiveness, defining success as achieving measurable parallel speedup while preserving byte-for-byte data integrity.

B. Workload and Benchmark Configuration

The primary workload for all benchmark sweeps is a $\sim$1.08GB file (1,082,774,528 bytes) produced by concatenating the Canterbury Corpus `large` reference tarball (11,162,624 bytes) 97 times, providing a realistic and compressible dataset. Against this input we swept thread counts $\{1, 2, 4, 8, 16, 32\}$ with block size 900 KB, compression level 9, and 3 repeats per configuration. Each AI-assisted stage was benchmarked in its own isolated Git worktree, and the lbzip2 and pbzip2 references were benchmarked on the same input and thread sweep.

C. Evaluation Metrics

The metrics tracked by the harness include:

- **Runtime:** Total compression time.
- **Throughput:** Input size divided by compression time, measured in MB/s.
- **Speedup:** Sequential runtime divided by parallel runtime.
- **Parallel Efficiency:** Speedup divided by number of threads.
- **Memory Usage:** Peak memory consumption.
- **Correctness:** Whether decompressed output matches the original file.

Performance improvements are quantified mathematically using the following standard definitions:

$$Speedup = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} \tag{1}$$

$$Throughput = \frac{Input\_Size}{T_{\text{compression}}} \tag{2}$$

V. Results

A. AI-Generated Implementations

All three AI-assisted stages parallelize the same sequential pbzx baseline, but they arrive at substantially different designs, which is what the rest of this section compares.

Stage 1 (naive) parallelizes the block-compression loop directly with OpenMP: a single `#pragma omp parallel for schedule(dynamic)` spreads the independent 900 KB blocks across threads in a fork/join model, the only explicit coordination being an atomic flag for error propagation. Each block allocates and frees its own libbz2 workspace.

Stage 2 (constraint-guided) instead builds the hand-written `pthread` pipeline the prompt mandated: a reader thread

loads raw blocks and assigns unique IDs, a pool of worker threads compresses them independently, and a separate writer thread emits the compressed blocks in original order. The three roles coordinate through one mutex-guarded, bounded queue (`p->mtx` with per-role condition variables), and buffers are allocated and freed per block.

Stage 3 (profiling-guided) keeps Stage 1's OpenMP fork/join loop but, guided by profiling, swaps the per-block workspace `malloc`/`free` for a per-thread bump-pointer arena — a 16 MB thread-local buffer allocated once and reset per block, with oversize requests falling back to `malloc` — eliminating the repeated large allocations that glibc was servicing through `mmap`.

In short, Stages 1 and 3 share an OpenMP fork/join structure (Stage 3 adding arena-based memory reuse), whereas Stage 2 uses an explicit reader/worker/writer `pthread` pipeline with per-block allocation. All results that follow use the workload and benchmark configuration of Section IV-B (the 1.08GB Canterbury Corpus input, the thread sweep $\{1, 2, 4, 8, 16, 32\}$, block size 900 KB, and compression level 9).

B. Correctness

All three AI-assisted stages produce **byte-identical compressed output** regardless of thread count, while lbzip2 differs only slightly in output size due to its block framing — not a correctness issue. Round-trip decompression against the original 1.08GB input passes for all five parallel implementations (the three AI-assisted stages plus the lbzip2 and pbzip2 references) at every thread count tested. This confirms that increasing thread count does not change the compressed output or break correctness for any of the three AI-generated parallel versions.

C. Runtime and Throughput

Table I reports mean compression time (seconds, averaged over 3 repeats) at each thread count.

TABLE I
MEAN COMPRESSION TIME (S) ON THE 1.08GB INPUT

Threads	Stage 1 (naive)	Stage 2 (constrained)	Stage 3 (profiling)	lbzip2 (reference)	pbzip2 (reference)
1	80.84	81.77	78.72	67.90	81.13
2	40.63	40.62	39.43	34.51	41.33
4	20.80	20.74	20.02	17.58	20.89
8	10.53	10.50	10.23	9.10	10.75
16	5.52	5.50	5.36	4.96	5.89
32	3.06	3.49	3.03	3.14	3.79

In absolute terms, **Stage 3 (profiling-guided) is the fastest of the three AI-assisted stages at every thread count**, and is competitive with — even marginally faster than — the mature lbzip2 reference at high thread counts (3.03s vs. 3.14s at 32 threads). Stage 1 (naive) and Stage 2 (constraint-guided) are otherwise nearly indistinguishable through the middle of the sweep (within hundredths of a second at 2–16 threads); Stage 2 differs mainly at the two ends, with the highest single-threaded baseline (81.77s) and the weakest 32-thread time (3.49s, vs. 3.03–3.06s for Stage 1 and Stage 3) — a gap

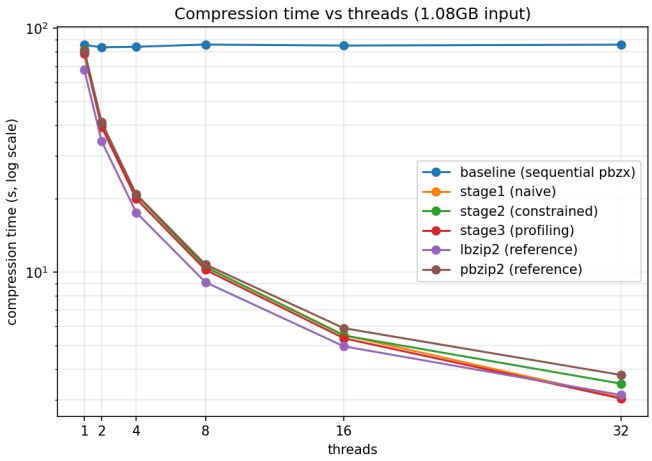

Fig. 1. Compression time vs. thread count (log-log scale) for all five parallel implementations and the sequential baseline on the 1.08GB input.

we trace to its synchronization design in Section VI-A. The second external reference, pbzip2, tracks the AI stages closely through 8 threads but falls behind from 16 threads onward, ending as the slowest implementation at 32 threads (3.79s).

Figure 2 shows the corresponding throughput curves, which mirror the runtime results: all five parallel implementations reach roughly 285–360 MB/s at 32 threads (Stage 3 highest at ≈359 MB/s, pbzip2 lowest at ≈286 MB/s), up from approximately 13–16 MB/s single-threaded.

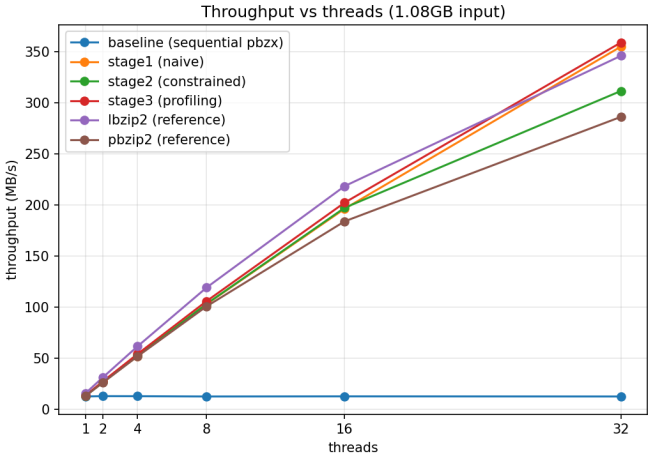

Fig. 2. Throughput (MB/s) vs. thread count for all five parallel implementations and the sequential baseline on the 1.08GB input.

D. Speedup

Table II reports speedup relative to each implementation's own single-threaded time, and Figure 3 plots the same data against an ideal linear-speedup reference line.

All three AI-assisted stages — along with both external references (lbzip2 and pbzip2) — track the ideal linear-speedup line closely up to 16 threads (∼2×, ∼4×, ∼8×, and ∼14–15× at each doubling), confirming that block-level parallelism

TABLE II
SPEEDUP RELATIVE TO EACH IMPLEMENTATION'S OWN 1-THREAD BASELINE

Threads	Stage 1 (naive)	Stage 2 (constrained)	Stage 3 (profiling)	lbzip2 (reference)	pbzip2 (reference)
1	1.00×	1.00×	1.00×	1.00×	1.00×
2	1.99×	2.01×	2.00×	1.97×	1.96×
4	3.89×	3.94×	3.93×	3.86×	3.88×
8	7.68×	7.79×	7.69×	7.46×	7.54×
16	14.64×	14.87×	14.69×	13.69×	13.77×
32	26.44×	23.43×	25.95×	21.64×	21.41×

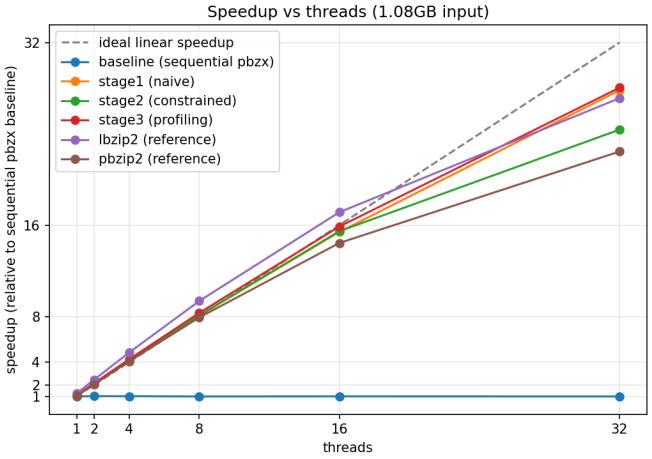

Fig. 3. Speedup vs. thread count relative to each implementation's own single-threaded baseline, with an ideal linear-speedup reference line (dashed).

is, as expected, embarrassingly parallel for a sufficiently large input (1,204 blocks at 900 KB each leaves ample work to keep 16 threads fed). Scaling tapers off at 32 threads for every implementation, plateauing around 21–26× — even though the machine has many more physical cores available (72). This suggests that the bottleneck is not simply core count, but more likely a combination of memory bandwidth, NUMA effects, block granularity, and synchronization overhead, consistent with the profiling-stage finding that performance becomes limited by factors beyond software-level parallelism at high thread counts.

Note that the two external references post the *lowest* relative speedups at 32 threads (pbzip2 21.41×, lbzip2 21.64×), but for different reasons. lbzip2's is low purely because its single-threaded baseline is already the fastest of all (67.90s vs. 78–82s for the AI stages) — it has comparatively less headroom to climb. pbzip2, by contrast, starts from one of the slowest single-threaded baselines (81.13s) yet still scales the worst in absolute terms, so its low relative speedup reflects a genuine efficiency loss rather than a fast starting point. In **absolute** terms (Table I, Figure 1), lbzip2 remains competitive with, but does not dominate, the AI-assisted stages: Stage 1 and Stage 3 match or marginally beat it at 32 threads, while pbzip2 is the slowest implementation at that point (3.79s).

VI. DISCUSSION AND ANALYSIS

Contrary to the concern that naive AI prompting may produce incomplete or unsafe parallelization, Stage 1 already produces a correct, near-linearly-scaling OpenMP implementation. This suggests that block-level parallelization of pbzx is a sufficiently well-isolated transformation that even a minimally-guided agent can identify and implement it correctly.

A. Root-Cause Analysis of AI Failures (Stage 2)

Constraint-guided prompting (Stage 2) produced correct but consistently slower code. Even under a fair elapsed-time comparison (excluding our timing instrumentation), Stage 2 is genuinely ∼9–12% slower than Stage 1 and Stage 3 at 32 threads, and — more tellingly — its run-to-run variance is far larger.

Ruling out the obvious suspects. Our instrumentation (`prof_lock`) showed that userspace `pthread_mutex` contention (the queue lock `p->mtx`) and worker idle time were *not* the cause — both are comparable across the three stages. The only remaining synchronization point that `prof_lock` cannot observe, yet is provably present, lies in the kernel: glibc's `malloc` services any allocation above its 128 KB threshold with `mmap`/`munmap`, and every `mmap`/`munmap` must hold the process-wide kernel `mmap_lock` (a reader/writer semaphore). This is the same mechanism behind the ∼674K page-faults and the `madvise`/`munmap`/`mmap`-dominated `strace` that motivated Stage 3's arena allocator.

Why Stage 2 specifically. Both the reader's 900 KB raw-block buffers and each worker's ∼7.6 MB BWT workspace exceed the 128 KB threshold, so all of them route through `mmap`/`munmap`. In Stage 1 and Stage 3, only the 32 worker threads perform these large allocations inside the timed region (and Stage 3's per-thread arena removes them almost entirely). Stage 2's mandated pipeline adds *two* more threads that allocate and free continuously for the whole run: the reader `malloc`/`free`s 900 KB raw buffers and the writer frees compressed-output buffers. These two threads contend for the same kernel `mmap_lock` as all 32 workers throughout execution, serializing memory operations and producing the jitter we observe as Stage 2's outsized variance.

The AI's failure. The agent faithfully implemented the prompt's structural mandate — dedicated reader and writer threads for ordered output — but could not foresee that these extra allocation sites would collide on a kernel lock invisible to userspace profiling. Over-constraining the architecture without performance feedback thus yielded a design that is correct and well-structured yet systematically slower and less predictable than the minimally-guided Stage 1.

B. Success of Profiling Feedback

Profiling-guided optimization (Stage 3) delivered the expected payoff. Built from the same sequential baseline as the other stages, but with profiling tools made available to the agent, it used empirical bottleneck data to drive targeted fixes — for example, replacing per-block `malloc`/`free` with

a per-thread bump-arena allocator dropped page-fault counts from ∼674K to ∼181K — measurably reducing both the single-threaded baseline and the absolute runtime at every thread count. This direct system feedback was critical to making it the fastest of the three AI stages overall and putting it on par with lbzip2.

C. Limitations

Several aspects of our evaluation limit how broadly the results should be generalized. First, all measurements use a single primary workload (the 1.08GB Canterbury Corpus input); we did not vary input content, size, or compressibility, so we cannot characterize behavior across different data profiles. Second, each configuration was run only three times, and we report means without variance or confidence intervals, so the small differences between stages at a given thread count should be read as indicative rather than statistically established.

Third, our thread sweep stops at 32 threads on a machine with 72 physical cores across two NUMA nodes. Scaling is near-linear up to 16 threads and begins to taper at 32 for all five implementations alike, which hints that the bottleneck is shifting away from raw core count toward memory bandwidth, NUMA effects, block granularity, or synchronization overhead. Without 64- and 72-thread runs, however, we cannot fully observe socket-level saturation or cross-socket NUMA behavior, so this remains a partial picture. Fourth, we did not control for or report the storage medium of the input and output files (SSD, RAM disk, tmpfs, or network storage); because compression is partly I/O-bound, both the absolute runtimes and the apparent I/O ceiling at high thread counts could shift on a different storage backend.

Finally, lbzip2 and pbzip2 are useful external references but not a strict apples-to-apples comparison. pbzip2 shares our libbz2 codec and differs mainly in its parallel runtime, whereas lbzip2 additionally replaces the core block-sorting algorithm with a faster suffix-array BWT. lbzip2's per-core advantage therefore reflects algorithmic optimization, not parallelization alone, and should not be read as a direct measure of parallel-scaling quality.

D. Additional Study: GPU Acceleration Inside libbz2

Beyond the CPU-side, block-level parallelization studied above, we conducted a follow-up experiment on a separate GPU branch to assess whether the *intra-block* stages of the standard libbz2 compression path could be offloaded to CUDA while preserving both the `.bz2` format and the public API.

Profiling the single-threaded path showed that block sorting dominated runtime, making it the natural first GPU target. We implemented three optimizations: (i) offloading `BZ2_blockSort` to CUDA; (ii) generating the BWT last column directly on the GPU (`BZ2_CUDA_BWT`), so the subsequent MTF stage reads the transformed block sequentially rather than through the random-access `block[ptr[i]-1]` pattern; and (iii) a CUDA-assisted Huffman table optimization (`BZ2_CUDA_HUFFMAN`) that parallelizes per-group code-cost evaluation and frequency accumulation, while leaving final bitstream emission on the CPU to preserve the stream format.

Table III summarizes results on the 1 GB input. The CPU fallback required 85.80 s, of which block sorting accounted for 60.97 s (76.6%). Offloading sorting to CUDA cut total time to 33.44 s; GPU-side BWT generation further reduced it to 29.95 s (MTF: 11.88 s → 8.11 s); and CUDA-assisted Huffman optimization brought it to 27.04 s (bitstream phase: 6.88 s → 4.18 s), a 3.17× end-to-end speedup over the fallback.

TABLE III
GPU EXTENSION RESULTS ON THE 1 GB INPUT

Configuration	Time (s)	Bottleneck after
CPU fallback	85.80	CPU block sort
+ CUDA block sort	33.44	CPU MTF / Huffman
+ GPU BWT	29.95	CPU MTF / Huffman
+ CUDA Huffman	27.04	MTF (sequential)

Not all stages benefited. An experimental CUDA MTF prototype, which computed MTF ranks on the GPU while leaving zero-run compaction on the CPU, preserved correctness but degraded performance severely: total time rose to 136.14 s, with the MTF phase alone reaching 114.41 s. This reflects the inherently sequential nature of MTF—each symbol depends on the evolving recency ordering of earlier symbols—so a naive parallel backward-scan incurs excessive repeated work and poor memory behavior on the GPU. The prototype was therefore removed from the final path.

The broader lesson mirrors the CPU study: selective acceleration is most effective when applied to phases with strong data parallelism (block sorting, table optimization), whereas accelerating one dominant phase merely migrates the bottleneck to the next, inherently sequential stage (here, MTF and final encoding), which resists efficient GPU implementation. Detailed hardware specifications for this GPU study are provided in Appendix E.

VII. Conclusion

This project evaluated how different levels of AI guidance affect the parallelization of a bzip2-like compression system. Across all three AI-assisted implementations, the generated code preserved correctness and achieved substantial speedup from block-level parallelism, showing that LLM agents can identify and implement well-isolated parallel transformations when the program structure exposes independent units of work.

The main finding, however, is that more explicit guidance does not necessarily lead to better performance. The naive OpenMP implementation in Stage 1 already scaled close to linearly up to 16 threads, while the constraint-guided Stage 2 introduced a more complex reader/worker/writer pipeline that became less stable and slower at high thread counts. In contrast, Stage 3 benefited from profiling feedback: by exposing actual bottlenecks such as allocation overhead and page-fault behavior, profiling allowed the agent to make targeted

optimizations, including per-thread arena reuse, and produced the best overall AI-assisted implementation.

These results suggest that AI-assisted performance optimization is most effective when the agent is guided by empirical system feedback rather than by rigid architectural prescriptions alone. Static constraints can help ensure correctness and structure, but they may also encode assumptions that are not performance-optimal on the target machine. Profiling, by contrast, gives the agent visibility into runtime behavior that is difficult to infer from source code alone. More broadly, our CPU and GPU experiments both show that optimization is an iterative bottleneck-shifting process: once one dominant cost is removed, the next limitation may come from memory behavior, synchronization, algorithmic structure, or inherently sequential stages.

ACKNOWLEDGMENT

REFERENCES

[1] "bzip2 and libbzip2." [Online]. Available: http://www.bzip.org/. [Accessed: Jun. 13, 2026].

[2] M. Burrows and D. J. Wheeler, "A block-sorting lossless data compression algorithm," Digital Equipment Corporation, Systems Research Center, Palo Alto, CA, USA, Tech. Rep. 124, May 1994.

[3] "pbzip2," *Compression*. [Online]. Available: https://compression.great-site.net/pbzip2/?i=1. [Accessed: Jun. 13, 2026].

[4] K. J. Nesbitt, "lbzip2: Parallel bzip2 utility," GitHub repository. [Online]. Available: https://github.com/kjn/lbzip2. [Accessed: Jun. 13, 2026].

[5] "bzip2," GitLab repository. [Online]. Available: https://gitlab.com/bzip2/bzip2. [Accessed: Jun. 13, 2026].

APPENDIX A
CLAUDE CLI PROMPT ARCHITECTURE

When Claude Code submits a request to the Anthropic Messages API, the payload is assembled from several distinct layers injected by the harness. The following annotated structure was derived from an intercepted first-turn API call during Stage 3 development.

System prompt (`system` array). Four `text` blocks are passed in the top-level `system` field. (1) A billing and telemetry header, not cached. (2) The agent identity declaration (*"You are Claude Code, Anthropic's official CLI for Claude."*), not cached. (3) The core operational instructions—tool descriptions, permission model, response-style rules, and session guidance—marked `cache_control: {type: "ephemeral", scope: "global", ttl: "1h"}`; the `scope: "global"` flag allows this block's KV cache to be shared across all Claude Code sessions globally, substantially reducing per-request latency and cost. (4) Session-specific context (auto-memory system instructions, current git status, environment facts, available skills), marked `cache_control: {type: "ephemeral", ttl: "1h"}` (session-scoped).

CLAUDE.md and auto-memory injected into the user turn. The project's `CLAUDE.md` and the per-project memory index (`MEMORY.md`) are *not* placed in the system prompt. Instead, Claude Code prepends them to the first user message as a separate `text` content item wrapped in a `<system-reminder>` XML tag. The actual user message appears as a second content item and carries its own `cache_control` annotation so subsequent turns reuse its cached representation.

Hook output as a `role: "system"` turn. A SessionStart hook fires before the first model turn; its output (the skills manifest, deferred-tool registry, and available agent types) is injected as a second entry in the `messages` array with `role: "system"`. This is a Claude Code harness extension: the standard Anthropic Messages API does not define a `"system"` role inside the `messages` array; Claude Code uses it to inject harness-side context between turns without attributing it to the user.

Tools, extended thinking, and context management. All tool definitions (Agent, Bash, Edit, Read, Write, Skill, ToolSearch, AskUserQuestion, ScheduleWakeup) are transmitted as full JSON Schemas on every request and are not cached. Extended chain-of-thought reasoning is enabled via `"thinking": {"type": "adaptive"}` and `"output_config": {"effort": "high"}`. A `context_management` edit of type `clear_thinking_20251015` strips completed thinking blocks from prior turns, reclaiming context-window space while preserving all final assistant responses.

The condensed, annotated payload structure is shown below.

```
{
  "model": "claude-opus-4-8",
  "system": [
    /* 1: billing/telemetry header -- not cached */
    { "type": "text",
      "text": "x-anthropic-billing-header:
    cc_version=...; cc_entrypoint=cli;" },
    /* 2: agent identity -- not cached */
    { "type": "text",
      "text": "You are Claude Code, Anthropic's
    official CLI for Claude." },
    /* 3: core instructions -- globally cached
    across all sessions, 1h TTL */
    { "type": "text",
      "text": "<tools, permissions, style rules,
    response guidelines ...>",
      "cache_control": { "type": "ephemeral", "
    scope": "global", "ttl": "1h" } },
    /* 4: session context -- session-scoped cache,
    1h TTL */
    { "type": "text",
      "text": "<git status, environment, auto-
    memory instructions, skills>",
      "cache_control": { "type": "ephemeral", "ttl
    ": "1h" } }
  ],
  "messages": [
    {
      "role": "user",
      "content": [
        /* CLAUDE.md + MEMORY.md prepended before
    the user's actual message */
        { "type": "text",
          "text": "<system-reminder>\n# claudeMd\n[
    CLAUDE.md contents]\n\n# memory\n[MEMORY.md
    index]\n</system-reminder>\n" },
        /* actual user input -- cached so later
    turns reuse this KV entry */
        { "type": "text",
          "text": "hi test 123",
```

```
          "cache_control": { "type": "ephemeral", "
    ttl": "1h" } }
      ]
    },
    /* SessionStart hook output -- injected as a
    non-standard "system" role */
    {
      "role": "system",
      "content": "<skills manifest, deferred tools,
     available agent types>"
    }
  ],
  "tools": [ /* full JSON Schema for each tool --
    re-sent every request */ ],
  "thinking":           { "type": "adaptive" },
  "output_config":      { "effort": "high"   },
  "context_management": {
    "edits": [{ "type": "clear_thinking_20251015",
    "keep": "all" }]
  },
  "max_tokens": 64000,
  "stream": true
}
```

APPENDIX B
TASK PROMPTS (CLAUDE.MD PER STAGE)

Each stage branch carried its own `CLAUDE.md`, injected into every API request via the `<system-reminder>` mechanism described in Appendix A. The three files illustrate the deliberate escalation of guidance across stages: Stage 1 is a minimal open-ended directive; Stage 2 adds explicit structural constraints and a hardware-discovery mandate; Stage 3 replaces the structural mandates with a profiling-tool allowlist and an iterative profile → optimize → verify loop instruction.

Stage 1 — Naive (branch `stage/1-naive`)

```
# pbzx Stage 1: Naive AI Parallelization

This is a bzip2 compression tool called pbzx.

## Your task

Please parallelize this bzip2 compression program
    using multiple threads.
Preserve correctness and improve performance.

## Source files you may modify

- src/main.c        -- main entry point and
    compression pipeline
- src/writer.c      -- writes compressed blocks to
    output
- src/bz_block.c/h  -- per-block bzip2 compression
- src/block_reader.c/h -- reads input into fixed-
    size blocks
- src/args.c/h      -- argument parsing (already
    accepts --threads N)
- Makefile          -- update CFLAGS here if needed
     (-fopenmp, -lpthread)

## Do NOT modify

- third_party/bzip2/   -- vendored libbz2 source
- libbz2.a             -- prebuilt static library

## Build

    make clean && make all   # produces ./pbzx

## Test correctness

    python3 bench/verify.py ./pbzx <input_file>
    # Expected: PASS  <input_file>

## Benchmark

    python3 bench/run_bench.py --pbzx ./pbzx --
    inputs <input_file> \
      --threads 1 2 4 8 --block-sizes 900000 --
    level 9 --repeat 3 \
      --out results/stage1_results.csv

## Interface

    ./pbzx -i input.bin -o output.bz2 [--threads N]
     [--block-size B] [--level L]

The --threads flag is parsed but currently unused
    in the compression step.
Output must be valid bzip2: bunzip2 -c output.bz2 |
     cmp - input.bin must succeed.
--threads 1 must behave identically to the original
     sequential code.
```

Stage 2 — Constraint-guided (branch `stage/2-constrained`)

```
# pbzx Stage 2: Constraint-guided AI
    Parallelization

This is a bzip2 compression tool called pbzx. Your
    goal is to parallelize it
using block-level parallelism, choosing design
    parameters informed by the actual
hardware this machine offers.

## Your task

Please parallelize the compressor using block-level
     parallelism.

Requirements:
1. Split the input file into fixed-size blocks.
2. Assign each block a unique block ID.
3. Use worker threads to compress blocks
    independently.
4. Use a writer thread (or equivalent ordered
    mechanism) to emit blocks in
   the original order.
5. Avoid race conditions and unnecessary global
    locks.
6. Avoid changing the compression result (
    decompressed output must match
   input exactly).
7. Before choosing thread count, block size, and
    queue depth, run lscpu,
   nproc, free -h, and getconf -a | grep -i cache
    to understand the machine
   hardware, then use those values as constraints
    in your implementation.

## Source files you may modify

- src/main.c        -- main entry point and
    compression pipeline
- src/writer.c      -- ordered output
- src/bz_block.c/h  -- per-block compression (
    thread-safe: no global state)
- src/block_reader.c/h -- block reader
- src/args.c/h      -- argument parsing (already
    accepts --threads N)
- Makefile          -- update CFLAGS as needed
```

```
- You may add new src/*.c / src/*.h files

## Do NOT modify

- third_party/bzip2/
- libbz2.a

## Build / Test / Benchmark

    make clean && make all
    python3 bench/verify.py ./pbzx <input_file>
    python3 bench/run_bench.py --pbzx ./pbzx --
    inputs <input_file> \
      --threads 1 2 4 8 --block-sizes 900000 --
    level 9 --repeat 3 \
      --out results/stage2_results.csv

## Interface

    ./pbzx -i input.bin -o output.bz2 [--threads N]
     [--block-size B] [--level L]

--threads 1 must still work correctly. Output must
    be valid bzip2.
```

Stage 3 — Profiling-guided (branch `stage/3-profiling`)

```
# pbzx Stage 3: Profiling-guided AI Parallelization

This is a sequential bzip2 compression tool called
    pbzx. Your goal is to
parallelize it and iteratively optimize it using
    profiling tools.

## Your task

Please parallelize this bzip2 compression program
    using multiple threads,
then use profiling tools to measure bottlenecks in
    your own implementation
and improve it. Repeat the profile -> optimize ->
    verify cycle until you
are satisfied with the performance.

## Permitted profiling tools

You have full permission to run any of these on ./
    pbzx at any time:

- perf stat -d ./pbzx ...                   --
    hardware counters
- perf record -g ./pbzx ... && perf report -- call-
    graph hotspots
- valgrind --tool=callgrind ./pbzx ...      --
    instruction-level profiling
- valgrind --tool=massif ./pbzx ...         -- heap
    profiling
- /usr/bin/time -v ./pbzx ...               -- wall
    time + peak RSS + CPU%
- strace -c ./pbzx ...                      --
    system call summary
- lscpu, nproc, free -h, getconf -a | grep -i cache
     -- machine info

## Source files you may modify

- src/main.c        -- main entry point and
    compression pipeline
- src/writer.c      -- writes compressed blocks to
    output
- src/bz_block.c/h  -- per-block bzip2 compression
- src/block_reader.c/h -- reads input into fixed-
    size blocks
- src/args.c/h      -- argument parsing (accepts --
    threads N)
- Makefile          -- update CFLAGS as needed
- You may add new src/*.c / src/*.h files

## Do NOT modify

- third_party/bzip2/
- libbz2.a

## Build / Test / Benchmark

    make clean && make all
    python3 bench/verify.py ./pbzx <input_file>
    # Expected: PASS  <input_file> -- verify after
    every optimization round
    python3 bench/run_bench.py --pbzx ./pbzx --
    inputs <input_file> \
      --threads 1 2 4 8 --block-sizes 900000 --
    level 9 --repeat 3 \
      --out results/stage3_results.csv

## Interface

    ./pbzx -i input.bin -o output.bz2 [--threads N]
     [--block-size B] [--level L]

--threads 1 must still work correctly. Output must
    be valid bzip2.
```

APPENDIX C
REPRESENTATIVE AGENT OUTPUT

The complete agent interaction for each stage—including the prompts, the agent's extended thinking, tool calls, and generated code—is preserved verbatim as a `chat-history.txt` file on the corresponding Git branch:

- **Stage 1 (naive):** https://github.com/xdh1129/AI-assisted-Parallelization-of-bzip2-Compression/blob/stage/1-naive/chat-history.txt
- **Stage 2 (constrained):** https://github.com/xdh1129/AI-assisted-Parallelization-of-bzip2-Compression/blob/stage/2-constrained/chat-history.txt
- **Stage 3 (profiling):** https://github.com/xdh1129/AI-assisted-Parallelization-of-bzip2-Compression/blob/stage/3-profiling/chat-history.txt

APPENDIX D
CHANGES MADE TO THE AGENT

The Claude Code agent was provided isolated Git worktrees and varying levels of command permissions via per-branch CLAUDE.md files. In Stage 2, it was granted permissions to run machine-discovery commands (lscpu, free, nproc). In Stage 3, the agent context was dynamically updated to include raw outputs from `perf stat` and `/usr/bin/time -v`, granting it full visibility into its own execution bottlenecks.

APPENDIX E
GPU ACCELERATION: EXPERIMENTAL SETUP AND EXTENDED RESULTS

This appendix details the hardware environment and extended results for the side-study on CUDA-based GPU acceleration discussed in Section VI-D.

A. Hardware and Software Specifications

The GPU offloading experiments were conducted on a dedicated SLURM-managed compute node equipped with the following hardware and software stack:

- **GPU:** NVIDIA Tesla V100 (SXM2 form factor) with 32GB HBM2 VRAM.
- **Host CPU:** Dual-socket Intel Xeon Gold 6154 @ 3.00 GHz (36 physical cores total, Hyper-Threading disabled).
- **Host Memory:** 754 GiB System RAM.
- **Software Stack:** NVIDIA Driver Version 535.161.08, CUDA Environment 12.2.